

Global AI Missions — Respuestas a las preguntas de la entrevista técnica

Prototipo "Mini Global AI Assessment": *The Last Train*, una misión narrativa que internamente es una evaluación de 10 ítems.
Repositorio: github.com/cayoUte/Global-AI-Missions

Respuestas a las 10 preguntas del §11 del enunciado. Cada una explica la decisión y termina con **dónde verla en el repositorio**: archivos, funciones y tests reales (los tests de pytest viven en `backend/tests/` y se citan como `archivo::test`). Detalle ampliado en `docs/DECISIONS.md` y `docs/INTERVIEW.md`.

1. ¿Qué cambia si pasamos de 100 a 100.000 estudiantes, y por qué?

La API ya es **sin estado**: la sesión es un JWT en cookie y todo el estado de la misión vive en PostgreSQL. Eso permite poner varias réplicas detrás de un balanceador sin tocar código. Hay una sola excepción, a propósito: el **rate limit de login** vive en memoria de cada proceso. Con una instancia es correcto; con varias, cada réplica contaría por su lado, así que pasa a Redis o al API gateway.

La base de datos es el cuello de botella natural. Primero pondría PgBouncer y réplicas de lectura para World, Progress y reportes. Los índices por usuario y fecha ya existen, y más adelante particionaría `attempt_steps`. El contenido de una misión es inmutable por versión, así que se cachea sin riesgo. SPA, fuentes y audio pregenerado van a un CDN.

La pieza lenta es el LLM, y ya está aislada. Corre en la **fase 2 del submit**, después del commit de la **calificación**, con 8 s de presupuesto y fallback determinista. A escala, esa costura se convierte en una cola con workers. El reporte ya se muestra aunque todavía no exista la fila de feedback. Encima de eso irían tope de gasto por alumno, métricas (p95, tasa de fallback, costo) y pruebas de carga antes de cada salto.

DÓNDE VERLO EN EL REPOSITORIO

- `backend/app/core/rate_limit.py`: el docstring declara que es el único estado por proceso.
- `backend/app/services/attempts.py::submit` (las dos fases) y `backend/app/services/feedback.py::ensure_feedback` (el coach, después del commit).
- `backend/app/services/content.py`: caché de versiones inmutables.
- `backend/app/repositories/db.py::get_engine` (pool con `pool_pre_ping`) y `backend/app/api/middleware.py` (`X-Request-ID`, logs con latencia).
- `tests/api/test_mission_flow.py::test_a_coach_failure_after_grading_still_returns_the_report`
- `docs/DECISIONS.md` §5.

2. ¿Cómo evitas que un estudiante vea las respuestas correctas en el navegador?

Las claves **nunca salen del servidor** mientras el intento está abierto. El frontend no tiene ninguna pregunta en el código: todo viene de la API. Durante la misión el cliente solo recibe un `StateView`, que se arma en una sola función **por lista blanca**. Copia el prompt, el estímulo y el id y texto de cada opción, nada más: ni `is_correct`, ni clave, ni explicación, ni pista, ni `skill / cefr`, ni aristas o nodos futuros. La respuesta a `answer` tampoco dice si acertaste; lo descubres por la historia. Los 409 devuelven el estado actual para resincronizar, pero sin corrección.

Esto está probado a máquina: un escáner recursivo recorre cada respuesta de partidas completas, incluidos los cuerpos de los 409, buscando claves prohibidas. Otro test revisa el código y el bundle de la SPA. La

explicación y la respuesta correcta aparecen solo en el Reporte, cuando el intento ya se envió.

Hay un límite que declaro: el guion de *listening* llega al navegador porque lo lee `speechSynthesis`. En producción serían audios pregenerados detrás de URLs firmadas.

DÓNDE VERLO EN EL REPOSITORIO

- `backend/app/services/state_view.py::to_state_view` y `_checkpoint_view` (la lista blanca).
- `backend/app/repositories/missions.py::get_answer_key`: el único camino explícito a la clave.
- `docs/contracts/api-contract.md` §9, con la lista de campos prohibidos.
- `tests/services/test_state_view.py::test_checkpoint_view_is_a_whitelist`,
`::test_no_forbidden_key_on_any_node`
- `tests/api/test_mission_flow.py::test_the_real_mission_runs_end_to_end_without_leaks`
- `tests/api/test_qa_gaps.py::test_conflict_bodies_of_an_open_attempt_leak_nothing`
- `tests/contracts/test_spa_has_no_answers.py::test_the_production_bundle_has_no_item_content_or_answer_keys`

3. ¿Dónde vive el progreso y cómo modelas los intentos?

Un intento es una ejecución de una misión y está atado a la **versión** de contenido en la que se jugó. Cada transición se guarda como una fila de `attempt_steps` con su padre, la acción, el costo acumulado y la decisión de Maya. Es literalmente el nodo de búsqueda de un libro de IA. Si sigues los punteros desde el final, obtienes el **Diario** del reporte. Las respuestas van en `attempt_answers`, con una única respuesta por checkpoint. Al enviar se escriben los totales, el nivel y los resultados por habilidad.

El progreso **no se guarda**: es una consulta que suma aciertos y total por habilidad en los **últimos 3 intentos enviados**, así que nunca se desincroniza. La memoria de Maya (`coach_memory`) es lo único acumulado, y solo sirve para el tono y el saludo, nunca para la nota.

DÓNDE VERLO EN EL REPOSITORIO

- `backend/app/models/attempts.py` (`Attempt`, `AttemptStep`, `AttemptAnswer`, `AttemptSkillScore`) y `docs/data/DATA_MODEL.md`, con la tabla "attempt_steps ↔ Search Node".
- `backend/app/engine/diary.py::build_diary`
- `backend/app/repositories/progress.py::get_profile`
- `tests/engine/test_simulator_and_diary.py::test_diary_rebuilds_the_path_from_parent_pointers`
- `tests/data/test_repositories.py::test_progress_of_the_veteran`
- `tests/api/test_postgres_concurrency.py::test_the_seeded_veteran_progress_over_http`

4. ¿Cómo agregas Pre-A1, A1, A2, B1, B2 y C1 sin rediseñar?

Los seis niveles ya existen como **datos**. `cefr_levels` es una tabla de referencia con un `rank`, los ítems la referencian por FK y los esquemas JSON ya aceptan `C1`. Las reglas que comparan niveles, como los desbloques `level_reached` o el tope por evidencia, usan ese `rank`, no condicionales por nivel.

Una misión B2–C1 nueva es **contenido**: se agrega al catálogo, se escriben `items.json` y `mission.json`, se valida y se hace seed. No hace falta migración. Una versión de contenido nueva no toca los intentos anteriores.

Lo que hoy vive en código, y lo digo claro, son las bandas de esta misión (tope B2) y el blueprint 4/2/2/2 del validador. Para un producto con muchas misiones, eso pasa a ser un campo de cada misión.

DÓNDE VERLO EN EL REPOSITORIO

- `backend/app/models/reference.py` (`CefrLevel`), `backend/alembic/versions/0001_initial.py` (siembra los niveles) y `docs/contracts/items.schema.json` (enum `cefr`).
- `backend/app/services/leveling.py` (`CEFR_ORDER`, `BANDS`, `suggest_level`) y `backend/app/services/world.py::unlock_rule_met`.
- `backend/app/engine/validator.py` (`BLUEPRINT`, `CEFR_SEQUENCE`).
- `docs/data/DATA_MODEL.md`, sección "Adding a mission, or C1 content, without schema changes".
- `tests/services/test_grading_leveling.py::test_worked_examples`,
`::test_a_level_with_no_items_is_never_met`
- `tests/data/test_seed.py::test_new_content_creates_a_new_version_and_old_attempts_keep_theirs`

5. Si reemplazas Claude por otro modelo, ¿qué código cambia?

Para otro modelo de Claude, **ninguno**: es la variable `ANTHROPIC_MODEL`. Para otro proveedor compatible con OpenAI (Groq, xAI Grok), **tampoco**: `COACH_PROVIDER=openai_compatible` más la URL base, la clave y el modelo en el `.env`. Para un proveedor con API propia, **un archivo** en `backend/app/ai/` que implemente `generate(context) → CoachFeedback` y **una línea** en `factory.PROVIDERS`.

El resto no se entera. El puerto `CoachProvider` define la entrada y una salida JSON estricta que valida Pydantic. El prompt es un archivo versionado y lo comparten todos los adaptadores. Si el modelo falla, tarda más de 8 s o devuelve algo inválido, se usa el mock determinista. Cada feedback guarda proveedor, modelo y versión de prompt para poder compararlos. Un test garantiza que ningún SDK se importe fuera de `app/ai`.

Cómo lo verifico: los adaptadores tienen tests con clientes HTTP falsos (sin red), y

`backend/scripts/coach_smoke.py` hace una llamada real con la clave configurada; con `--list-models` muestra qué modelos admite esa clave. La nota nunca depende de que el LLM responda.

DÓNDE VERLO EN EL REPOSITORIO

- `backend/app/ai/ports.py` (`CoachProvider`, `CoachFeedback`), `backend/app/ai/factory.py` (`PROVIDERS`, `create_provider`) y `backend/app/ai/prompts/maya_feedback_v1.md`.
- `backend/app/ai/anthropic_provider.py`, `backend/app/ai/openai_compatible_provider.py` y `backend/app/ai/mock_provider.py`.
- `tests/ai/test_factory_and_seam.py::test_no_provider_sdk_is_imported_outside_app_ai`,
`::test_factory_selects_by_coach_provider`,
`::test_real_settings_accept_any_provider_name_and_boot_with_the_mock`
- `tests/ai/test_openai_compatible.py::test_submit_seam_with_groq_is_ready`,
`::test_output_that_breaks_the_contract_falls_back`
- `docs/ai/AI_ARCHITECTURE.md §2`.

6. ¿Cómo separas los permisos de estudiante, docente y administrador?

El rol está en el usuario (`users.role`, con un CHECK). El JWT solo identifica: en cada petición cargo el usuario desde la base, así que un cambio de rol o un usuario borrado aplican de inmediato. Los permisos son **dependencias de FastAPI**: `Student` o `TeacherOrAdmin` en la firma del endpoint. Un rol equivocado recibe `403 FORBIDDEN_ROLE`.

Dentro de un rol está el **alcance**. Un estudiante solo ve sus intentos: uno ajeno da 404, no 403, para no revelar que existe, y los ids son UUID. Un docente solo ve las clases que enseña (`classes.teacher_id + class_members`); otra clase también da 404. El admin ve todo, y en producción sería quien publica versiones de contenido. La vista docente del frontend es de solo lectura, y el RBAC se prueba por la API.

DÓNDE VERLO EN EL REPOSITORIO

- `backend/app/api/deps.py::require_role (Student, TeacherOrAdmin)` y `backend/app/services/auth.py::session_user`.
- `backend/app/services/world.py::_teacher_scope` y `::class_progress`; `backend/app/services/attempts.py::get_owned_attempt`.
- `tests/api/test_attempt_security.py::test_teacher_rbac`, `::test_student_endpoints_need_a_session_and_the_student_role`, `::test_another_students_attempt_is_404_everywhere`
- `tests/api/test_auth.py::test_the_role_comes_from_the_database_not_the_token`

7. ¿Qué pasa si el estudiante pierde la conexión a mitad de la evaluación?

No pierde nada, porque **el servidor es la fuente de verdad**. Cada `advance` y cada `answer` se guardan en una transacción corta, y el estado vive en la base. Si una petición falla por red, el reproductor la conserva y ofrece "Try again". Mientras tanto el checkpoint sigue bloqueado para que no haya un doble envío, y un aviso muestra que no hay conexión. Al volver la red, el cliente relee `GET /api/attempts/{id}` y sigue desde ahí.

Si recarga o entra desde otro dispositivo, "Start" devuelve el **intento abierto** en lugar de crear otro. Hay un solo intento abierto por alumno y misión, garantizado por un índice único parcial. Si la sesión expira, vuelve a entrar y retoma el mismo nodo. Una partida terminada pero no enviada sigue abierta y se puede enviar después. Si reintenta algo que ya había llegado, el 409 trae el estado actual y el cliente lo muestra sin error.

Límite conocido: si recarga justo en la escena de consecuencia, puede perder la frase de reacción de Maya (PD-032).

DÓNDE VERLO EN EL REPOSITORIO

- `backend/app/api/routers/attempts.py::get_attempt` y `::start_attempt`; índice `uq_attempts_one_open` en `backend/app/models/attempts.py`.
- `frontend/src/features/mission/MissionPlayerPage.tsx`: relectura al reconectar y resincronización con 409.
- `frontend/src/features/mission/playerReducer.ts` y `frontend/src/components/OfflineBanner.tsx`.
- `tests/api/test_qa_gaps.py::test_a_new_client_session_resumes_the_same_node_and_state`
- `tests/data/test_schema.py::test_one_open_attempt_per_student_and_mission`
- `frontend/src/features/mission/playerReducer.test.ts` > "keeps the failed request for Try again and stays on the same node (locked)", > "renders the server's truth on a 409 resync, silently"
- `frontend/e2e/resilience.spec.ts` > "two tabs, a reload and no speech: the server state wins, nothing breaks"

8. ¿Cómo garantizas que un estudiante no pueda modificar su puntaje desde el navegador?

No existe ningún endpoint que acepte un puntaje. El cliente solo dice "sigo desde el nodo X" o "mi respuesta en X es Y". Cada modelo de request usa `extra="forbid"`, así que un `is_correct` o un campo inventado da 422. `submit` **no lee cuerpo**: si mandas `{"score_pct": 100}`, se ignora y la nota sigue siendo la del servidor.

Cada respuesta se califica en el servidor y se **bloquea** en el primer envío, con una restricción UNIQUE más un lock de fila; una segunda respuesta da 409. El `submit` califica una sola vez, en la fase 1 y bajo lock, a partir de las respuestas guardadas. Repetirlo o hacerlo en paralelo devuelve el mismo reporte sin recalcular. Las pistas y el LLM no tocan la nota: el coach corre después del commit y su salida se valida.

DÓNDE VERLO EN EL REPOSITORIO

- backend/app/schemas/common.py::RequestModel (extra="forbid"), backend/app/schemas/attempts.py y backend/app/api/routers/attempts.py::submit (sin parámetro de cuerpo).
- backend/app/services/attempts.py::answer, ::submit y ::_grade_and_level; backend/app/services/grading.py.
- tests/api/test_attempt_security.py::test_the_client_can_never_send_a_score, ::test_answer_shape_errors_are_422_and_do_not_lock, ::test_a_second_answer_is_409_checkpoint_locked_with_the_current_state
- tests/api/test_postgres_concurrency.py::test_two_concurrent_answers_lock_the_checkpoint_once, ::test_two_concurrent_submits_make_one_report_and_one_memory_update
- tests/services/test_grading_leveling.py::test_hints_never_change_the_score
- tests/api/test_qa_gaps.py::test_a_valid_ai_provider_is_ready_and_still_cannot_change_the_grade

9. ¿Qué pruebas automatizadas harías primero y por qué?

Primero pruebo lo que haría **no confiable a la evaluación**, después lo que haría **fallar la demo**, y después el resto. El orden sale de docs/qa/TEST_PLAN.md §2:

1. **Integridad de la nota:** calificación, porcentaje por habilidad, conteos y nivel. Son funciones puras, baratas y exhaustivas. Reproduzco los 9 ejemplos del ASSESSMENT_SPEC.
2. **Fuga de claves:** el escáner recursivo sobre partidas completas y sobre los 409, más el bundle de la SPA. Una fuga no se ve en la UI; solo la atrapa una máquina.
3. **Manipulación desde el cliente:** extra="forbid", doble respuesta, nodo equivocado, submit antes del final.
4. **Propiedad y roles:** 404 para intentos y clases ajenos, 401/403 por rol.
5. **Invariantes del motor:** los 1.024 caminos mantienen 10 checkpoints y el blueprint 4/2/2/2, h exacta, un solo rescate.
6. **Concurrencia e idempotencia:** doble Confirm, dos pestañas, submits concurrentes sobre PostgreSQL real.
7. **Reanudación** tras perder la conexión.
8. **Fallback de IA:** timeout, clave inválida, JSON roto; el reporte siempre sale y la nota no cambia.
9. **El camino del evaluador** en E2E, solo con teclado, a 1280 y 360 px.

Hoy son 322 tests de pytest, 45 de Vitest y 10 E2E de Playwright (más 1 omitido).

DÓNDE VERLO EN EL REPOSITORIO

- docs/qa/TEST_PLAN.md §2 (estrategia por riesgo) y §5 (matriz de trazabilidad).
- tests/services/test_grading_leveling.py::test_worked_examples
- tests/api/test_qa_gaps.py::test_conflict_bodies_of_an_open_attempt_leak_nothing
- tests/engine/test_real_mission_invariants.py::test_all_1024_paths_visit_the_same_10_checkpoints_with_the_4_2_2_2_blueprint
- tests/api/test_qa_gaps.py::test_a_broken_ai_provider_never_breaks_submit_or_changes_the_grade
- frontend/e2e/mission.spec.ts > "new student: check-in → full mission by keyboard → report → progress"

10. ¿Qué priorizarías para producción en tres meses?

En este orden:

1. **Validez de la evaluación:** un banco de ítems con variantes y calibración, y después una prueba adaptativa. Eso cierra el límite de "repetir la misión no es reevaluar" y abre niveles hasta C1 con bandas por misión.
2. **Seguridad y operación:** rate limit compartido, refresh tokens, CSP, fuentes propias, audio pregenerado detrás de URLs firmadas (resuelve la fuga del guion de *listening*), auditoría, privacidad de menores, la cola del coach, observabilidad y pruebas de carga.
3. **Speaking** con reconocimiento de voz.
4. **Herramienta de autoría** de misiones que corra el validador del grafo antes de publicar.
5. **Paneles docentes y recordatorios o acompañamiento** a partir de la memoria de Maya, con un conjunto de evaluación para elegir proveedor de LLM.

DÓNDE VERLO EN EL REPOSITORIO

- docs/DECISIONS.md §8.
- docs/ai/AI_ARCHITECTURE.md §4 (hoja de ruta de IA).
- backend/app/engine/validator.py : la base de la herramienta de autoría.

Anexo · Límites conocidos y cómo se resuelven en producción

Límite	Por qué se aceptó	En producción
El guion de <i>listening</i> llega al navegador (<code>speechSynthesis</code> ; sin voz se muestra como texto, PD-014)	No había audio grabado en 8 h	Audio pregenerado detrás de URLs firmadas de vida corta
Repetir la misión después de ver su reporte no es una reevaluación segura (PD-020)	Una sola misión con 10 ítems fijos	Banco de ítems con variantes y prueba adaptativa
El rate limit de login vive en memoria de cada proceso (<code>backend/app/core/rate_limit.py</code>)	Correcto con una instancia	Redis o el API gateway
El coach usa el mock si no hay clave; la live demo usa Groq (plan gratuito) con respaldo automático	Demo reproducible, sin costo	<code>backend/scripts/coach_smoke.py</code> con una clave real; conjunto de evaluación
Fuentes desde Google Fonts (<code>frontend/index.html</code> , PD-032)	Rapidez del MVP	Fuentes autoalojadas en nuestro origen o CDN
Sin refresh token: la sesión dura 60 min	Simplicidad	Refresh token con rotación; la misión ya está a salvo en el servidor
Speaking no se puntúa; la vista docente es solo lectura y <code>/simulate</code> existe solo en modo demo	Alcance de 8 horas (PD-017, PD-026)	Ver la pregunta 10